Personalized PageRank with View-Count Edge Weights vs Implicit Matrix Factorization for TV Programme Recommendation on sweet.tv

Student: Vladislav Kutranov **Team:** Individual
Email: vvkutranov@edu.hse.ru **Date:** 2026-10-25
Assignment: Final Exam Quiz (AVITO) — TV Program Recommendation on sweet.tv
GitHub repository: https://github.com/elebrane/ML---RS **Application:** https://elebrane.github.io/ML---RS/
Designated Kaggle submission: `submission_ppr.csv`, public MAP@5 0.41930 (Fig. 2)

Abstract

The task is to predict, for 1,260 sweet.tv viewers, the five TV programmes they will watch in weeks 31–42, from a channel-level view log in which only 10.8% of schedule rows carry a programme id. Views were joined to labelled airings by channel and time overlap (watched = at least 80% of the airing), and implicit matrix factorization (MF) and Personalized PageRank (PPR) were compared with a popularity baseline on one temporal hold-out (fit weeks 11–20, evaluate weeks 21–30) with MAP@5; content-based filtering and association rules were also tried. PPR with edges weighted by the number of full views was best on validation (MAP@5 0.3595 vs 0.2652 for popularity) and on Kaggle (public MAP@5 0.41930, private 0.40867). The gain comes from repeat viewing: 92.8% of PPR's slots are programmes the user already watched, so PPR is the strongest "continue watching" ranker here, not a discovery engine.

Index Terms *— recommender systems, personalized PageRank, implicit feedback, TV programme recommendation, MAP@5.*

1. Introduction

Problem. The sweet.tv log has 5,821,987 viewing intervals (`user, channel, start, stop`) of 4,914 users in weeks 11–30 of 2020, plus broadcast schedules for weeks 11–42. Items are programmes (`tv_show_id`), not channels, so a view becomes an interaction only after it is matched to a labelled airing. For each of the 1,260 test users we must submit five programme ids for weeks 31–42; Kaggle scores them with MAP@5 [8].

Motivation. Linear TV differs from a catalogue: a programme can be watched only while it airs, and viewers return to the same news and shows every week. A useful method must survive the forward time shift and the sparse labelling, and its list must be judged against what a platform shelf would show.

Concrete example. Heavy viewer 15942687135322962396 (179 full views in weeks 11–20) watched 'ТСН. Неделя' 58 times and 'Завтрак с 1+1' 52 times in training. His actual top-5 in weeks 21–30 were these two plus three talk/entertainment shows; the PPR list contains four of the five (AP@5 = 0.76, Table 5).

Contributions.

- One leakage-safe protocol (join, ground truth, candidate pool, repeat policy, shared fallback, Top-5 rule) applied unchanged to popularity, MF and PPR (and to two secondary methods), in a browser application (TensorFlow.js for MF and PPR) that rebuilds the numbers and the Kaggle files step by step.
- Evidence that edge weights decide PPR quality on this data: weighting user–programme edges by full-view counts raises validation MAP@5 from 0.269 (binary edges, the homework-6 setting) to 0.3595. Weighted edges were not part of the practical classes; they were introduced in lecture 06 (slide 3, "Weight: strength of a connection") [9].

2. Related Work

Hu, Koren and Volinsky [1] treat implicit feedback as a binary preference with a confidence $c = 1 + \alpha r$ and fit all user–item cells; our MF uses this objective, optimised with Adam [5]. Jeh and Widom [3] personalise PageRank by restarting the walk at a user-specific vector; our PPR restarts at the user node of a bipartite graph, as in lecture 06 [9]. Apriori [2] and content-based profiles [4] underlie our two secondary methods. Melentev [6] describes the second-place solution of the original sweet.tv competition: candidates from a most-watched baseline and LightFM, re-ranked by LightGBM with watch-count and recency features, private MAP@5 0.42905. TensorFlow.js 4.22.0 [7] is the tensor runtime.

Alternatives considered. (i) A two-stage candidate + gradient-boosting re-ranker [6]: strongest known approach, but it needs a feature pipeline beyond the methods studied here — kept as the next step. (ii) Excluding already-watched programmes, as in homeworks 5–6: rejected because 56.7% of the cohort's actual top-5 programmes had been watched in weeks 11–20 and the competition does not forbid repeats. (iii) An absolute-seconds watch threshold: rejected because labelled airings mostly last 5 to 140 min (P5 300 s, P95 8,400 s), so a fixed cut treats a 5-minute and a 2-hour airing alike.

3. Method

Approach. Python prepares interactions once (the 5.8 M-row join); everything else runs in the JavaScript of the browser application; the long hyper-parameter sweeps were run in batch with the same code. Each method returns a score for every candidate programme of the target window; one Top-5 function turns scores into exactly five ids.



Fig. 1. Pipeline. The target-window schedule is used only to know what airs; viewing in the target window is never read for fitting.

Join and watched threshold. A view is matched to every labelled airing of the same channel (`vsetv_id = channel_id`) whose interval overlaps it; overlapping seconds are summed per (user, airing), capped at the airing duration and divided by it (screen fraction). The distribution is bimodal: of 3,014,129 user–airing pairs, 1.15 M have a fraction below 0.1 (zapping) and 0.53 M lie in [0.9, 1.0]. We call an airing watched at ≥ 0.8, the organiser's own rule for the ground truth (Data Description), which keeps 615,979 pairs (20.4%). Rows with `tv_show_id = 0` are unlabelled airtime and are dropped as items (never scored or recommended): 958,622 of 1,074,235 schedule rows in weeks 11–20, and only 52.4% of all viewing time overlaps a labelled airing. Schedule dates are day-first (`09.03.2020 06:00:00`) and are parsed with an explicit format; shifting the log by −4…+4 h confirmed that offset 0 aligns view starts with airing starts best.

Ground truth, split and candidates. Ground truth follows the organiser's rule: per user, count watched airings per programme in the target weeks; users with ≥ 5 programmes keep their 5 most frequent (ties: watched seconds, then id). Validation fits on weeks 11–20 and evaluates on weeks 21–30 (4,030 users have a top-5); the primary cohort is the 1,150 of them that are also Kaggle test users, and all 4,030 are reported as a robustness column. The final model is refit on weeks 11–30. Candidates are programmes airing in the target window (6,376 in weeks 21–30; 6,642 in weeks 31–42). Repeats are allowed for every method. Users with no watched labelled airing get the fallback: distinct viewers over the last 4 training weeks (the baseline's selected setting); it also fills any slot a method cannot fill.

From scores to exactly five ids. Scores of different methods are not comparable probabilities, so each list is built inside its own method: candidates with a finite score are sorted by score (ties: fallback rank, then id), the first five are taken, and missing slots are filled from the fallback list, skipping duplicates.

Method	Score of candidate programme *i* for user *u* (training weeks only)
Popularity (baseline)	Number of distinct users with a full view of *i* in the last 4 training weeks; same list for everyone.
Implicit MF [1], [5]	$u_u \cdot v_i$; loss $\Sigma_{u,i}\, c_{ui}(p_{ui} - u_u \cdot v_i)^2 + \lambda(\|U\|^2 + \|V\|^2)$ over all cells, $p = 1$ if watched, $c = 1 + \alpha \cdot$(full views). TF.js, Adam; mini-batches of users as in HW5 (L2 on the batch's rows plus its share of $\|V\|^2$).
PPR [3], [9]	Bipartite graph, edge weight w_{ui} = full views; transition = row-normalised weights ($P_{ij} = A_{ij}/\Sigma_k A_{ik}$, lecture 06 slide 5). $p \leftarrow \alpha P^{\mathrm{T}} p + (1 - \alpha) v_u$, α = link-following probability, v_u = one-hot at the user; stop at L1 change < 10^{-6} or 500 iterations; score = programme block of p. Batched two-block TF.js matMul, as in week 6.
Own history (reference)	Number of full views of *i* by *u* in training. Not a method: a no-model check of how much repetition alone explains.

Table 1. Methods. Only programmes with at least one watched airing in training get an MF or PPR score.

Secondary methods. Content-based filtering (cosine between a view-weighted user profile and programme vectors built from category, genres, people, channel and time slot [4]) and association rules (Apriori on weekly viewing baskets, "next-week" rules A → B [2]) were also implemented and tuned with the same protocol (`js/content.js`, `js/rules.js`; not shown in the application). Their results were weaker than PPR's (Section 4.2), so the report concentrates on MF and PPR.

Tools. Python 3.10.12 with pandas 2.3.3 and numpy 2.2.6 (join, exploration); JavaScript with TensorFlow.js 4.22.0 [7] (WebGL in the browser; WASM backend for the batch sweeps); d3 7.9.0 for the graph view. **Key design decisions:** (1) one protocol object (candidates, fallback, Top-5, MAP@5) used by every method, so differences come from scores only; (2) MF optimises the full user × programme matrix with `tf.matMul` instead of gathered embeddings (Section 5.2); (3) PPR edge weights are a tuned setting (binary, log(1+*n*), *n*). **Configuration.** Hyper-parameters were selected on validation only (grids in Table 2, column "Settings tried"). Seeds: MF initialisation 42/43, user permutation and batch order Mulberry32(seed+7 / +13); all other methods are deterministic, and a refit of every selected setting reproduced its MAP@5 exactly. Environment of the saved run: Linux arm64 VM on the author's Mac, 4 vCPU, 3.8 GB RAM, no GPU.

4. Experiments

4.1 Setup and data accounting

5,821,987 log rows; 10.8% of schedule rows in weeks 11–20 labelled (115,613 of 1,074,235), yet 52.4% of viewing time falls on labelled airings because they are long, popular programmes. Validation: 4,030 users with a weeks 21–30 top-5; cohort 1,150; 10 cohort users have no watched airing in weeks 11–20 and receive only the fallback (50 padded slots for most methods). Test: 1,260 users, 4 of them without any watched labelled airing in weeks 11–30 (cold start, fallback list). Of the 6,642 programmes airing in weeks 31–42, 2,855 were watched by someone in weeks 11–30; the other 3,787 have no co-viewing signal at all. Metric: MAP@5 as in the course slides, AP@5 = Σ precision at relevant ranks / min(5, $|R_u|$).

4.2 Results

Method	Selected setting	Settings tried	MAP@5 cohort 1,150	MAP@5 all 4,030	Padded slots	Valid. run, s	Test refit, s	Kaggle public / private
Popularity (required baseline)	distinct viewers, last 4 training weeks	6	0.2652	0.2398	0	0.6	1.5	–
Implicit MF (TF.js)	k 64, λ 50, α 1, count, Adam 0.05, 100 epochs, batch 1,024	12	0.3165	0.2905	50	133.2	59.4	0.37819 / 0.36688[a]
Personalized PageRank (TF.js)	**α 0.7 (link-following), edge weight = full-view count**	**15**	**0.3595**	**0.3237**	**50**	**55.1**	**97.1**	**0.41930 / 0.40867**
Own viewing history (reference)	*rank own weeks 11–20 programmes by full views*	*1*	*0.3552*	*0.3174*	*116*	*0.0*	*0.1*	–

Table 2. Validation (fit weeks 11–20, evaluate weeks 21–30) and Kaggle results; `results/summary.csv`, `results/kaggle/kaggle_scores.csv`. Run time = fit + scoring of the selected setting on the cohort (batch run, TF.js WASM backend, 4 vCPU). The grey row is a reference, not a method. [a] Both Kaggle files were built in the browser application; the MF file used the full-batch setting (validation MAP@5 0.3160), see 4.3.

Comparison vs baseline. On paired per-user AP@5 differences over the 1,150 users, PPR beats popularity by +0.094 MAP@5 (standard error 0.006) and MF by +0.043 MAP@5 (0.004). The secondary methods were less convincing: content-based filtering (MAP@5 0.2553) stayed below popularity, and association rules (MAP@5 0.3172) only matched MF, 0.042 MAP@5 below PPR (all rows in `results/summary.csv`). The no-model own-history list reaches MAP@5 0.3552: PPR's advantage over it, +0.004 MAP@5 (0.003), is within noise. On Kaggle, PPR (public MAP@5 0.41930) leads MF (0.37819) by 0.041 MAP@5, the same order and a similar gap as on validation (0.044 MAP@5 for that MF setting). Absolute Kaggle values are higher. The final model is fit on 20 weeks instead of 10, and history length matters: with the target fixed to weeks 21–30, the own-history ranking reaches MAP@5 0.234, 0.301 and 0.340 with 2, 5 and 10 weeks of history (`tools/history_length_check.js`). The hidden labels also cover other weeks and users, and the public and private halves alone differ by 0.011. For reference, the second-place solution of the original competition scored private MAP@5 0.42905 [6] against our 0.40867.

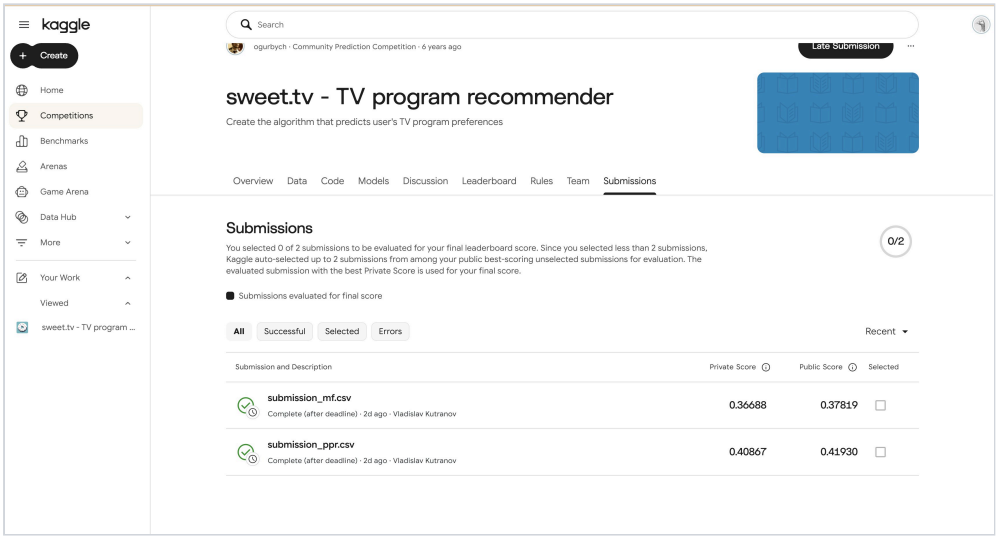


Fig. 2. Kaggle "Submissions" page (unmodified screenshot, 2026-10-25): `submission_ppr.csv` public 0.41930 / private 0.40867 and `submission_mf.csv` public 0.37819 / private 0.36688, both late submissions.

4.3 What the settings show

Edge weight \ α	0.15	0.3	0.5	0.7	0.85
binary (HW6 setting)	0.2712	0.2709	0.2702	0.2691	0.2641
log(1 + n)	0.3557	0.3554	0.3574	0.3524	0.3284
n = full views	0.3564	0.3564	0.3587	**0.3595**	0.3334

Table 3. PPR validation MAP@5 (cohort) by edge weight and link-following probability α; selected cell shaded.

PPR (Table 3, validation MAP@5): weights matter far more than α; binary edges forget that a user watched the news 58 times and a film once. **MF** (validation MAP@5): λ 10 / 50 / 100 → 0.2554 / 0.3140 / 0.2890 (k 32); α 5 lowers it to 0.2525; full batch 0.3160 vs 1,024 users × 100 epochs 0.3165 vs 20 epochs 0.2918.

4.4 Verification

- **Automated checks.** 18 checks pass in the application ("Run tests"): hand-computable fixtures (AP@5 slide example, rule support/confidence/lift, week-6 PPR reference values, cosine), data and leakage checks (no training row after week 20), and numerical checks of the TF.js MF and PPR with no leaked tensors; `tests/test_lib.py` checks the join on a fixture.
- **Re-run checks.** Every selected setting was refit and re-scored from scratch for the list analysis (Table 4): every MAP@5 value reproduced to six decimals. A browser validation run of the selected PPR setting gave MAP@5 0.359528, the value in Table 2. The uploaded `submission_ppr.csv` and `submission_mf.csv` are the files exported by the application's step-by-step builder (identical MD5).

Hand computation (by the author, on a calculator; user 15942687135322962396, PPR). Full views in weeks 21–30 (`results/analysis/handcheck_15942687135322962396.csv`):

id	title	full views	minutes	GT rank
2400480	'ТСН. Неделя'.	28	404	1
20088	'Завтрак с 1+1'. Информационн…	20	783	2
500281	Ток-шоу 'Говорит Украина'.	7	337	3
500315	Ток-шоу 'Касается каждого'.	5	579	4
500331	'Один за всех'.	5	418	5
200337	'Утро с 'Интером'.	5	212	
2400508	Сегодня. Итоги с Олегом Панют…	5	97	

Four programmes tie at 5 full views; the minutes tie-break keeps 500315 (579 min) and 500331 (418 min), so the ground truth is 2400480, 20088, 500281, 500315, 500331. The PPR list is 2400480, 20088, 500281, 240081, 500331: hits at ranks 1, 2, 3, 5. AP@5 = (1/1 + 2/2 + 3/3 + 4/5) / min(5, 5) = 3.8 / 5 = 0.76, equal to `ap5` in `results/analysis/per_user_ap_valid.csv`. The mean of the 1,150 PPR rows of that column, i.e. the MAP@5, is 0.359528, equal to `results/summary.csv`.

4.5 Recommendation examples (same two users for every method)

Users were chosen by a fixed rule before looking at any list: the cohort users at the 90th (heavy, 179 full views) and 10th percentile (light, 17 full views) of training activity. Green = programme in the user's actual weeks 21–30 top-5; ★ = not fully watched by this user in weeks 11–20 (new); Categories: Инфо = news/info, Развлечения = entertainment, Сериалы = series, Фильмы = films, Познавательное = factual. Titles are as in the schedule (long ones shortened).

#	Popularity	MF	PPR	Actually watched, weeks 21–30
1	**2400480** *Инфо* 'ТСН. Неделя'.	**2400480** *Инфо* 'ТСН. Неделя'.	**2400480** *Инфо* 'ТСН. Неделя'.	**2400480** *Инфо* 'ТСН. Неделя'.
2	**240081** *Инфо* Факты. День.	**20088** *Развлечения* 'Завтрак с 1+1'. Информационн…	**20088** *Развлечения* 'Завтрак с 1+1'. Информационн…	**20088** *Развлечения* 'Завтрак с 1+1'. Информационн…
3	**2400467** *Инфо* 'Викна-Новости'.	**2400467** *Инфо* 'Викна-Новости'.	**500281** *Развлечения* Ток-шоу 'Говорит Украина'.	**500281** *Развлечения* Ток-шоу 'Говорит Украина'.
4	**2400508** *Инфо* ★ Сегодня. Итоги с Олегом Панют…	**240081** *Инфо* Факты. День.	**240081** *Инфо* Факты. День.	**500315** *Развлечения* Ток-шоу 'Касается каждого'.
5	**20088** *Развлечения* 'Завтрак с 1+1'. Информационн…	**200344** *Развлечения* 'Чистоnews'.	**500331** *Развлечения* 'Один за всех'.	**500331** *Развлечения* 'Один за всех'.
AP@5	**0.28**	**0.40**	**0.76**	

Table 5a. Heavy viewer 15942687135322962396.

#	Popularity	MF	PPR	Actually watched, weeks 21–30
1	**2400480** *Инфо* ★ 'ТСН. Неделя'.	**2400508** *Инфо* Сегодня. Итоги с Олегом Панют…	**2400508** *Инфо* Сегодня. Итоги с Олегом Панют…	**2400508** *Инфо* Сегодня. Итоги с Олегом Панют…
2	**240081** *Инфо* ★ Факты. День.	**2400467** *Инфо* 'Викна-Новости'.	**400357** *Развлечения* 'Холостяк', 10 сезон, 13 эп.…	**5200303** *Познавательное* ★ 'Следствие вели...' с Леонидо…
3	**2400467** *Инфо* 'Викна-Новости'.	**2400480** *Инфо* ★ 'ТСН. Неделя'.	**2400480** *Инфо* ★ 'ТСН. Неделя'.	**90079798** *Фильмы* ★ Х/ф 'Очень плохие мамочки'.
4	**2400508** *Инфо* Сегодня. Итоги с Олегом Панют…	**240081** *Инфо* ★ Факты. День.	**2400467** *Инфо* 'Викна-Новости'.	**240081** *Инфо* ★ Факты. День.
5	**20088** *Развлечения* ★ 'Завтрак с 1+1'. Информационн…	**20088** *Развлечения* ★ 'Завтрак с 1+1'. Информационн…	**240081** *Инфо* ★ Факты. День.	**90045590** *Фильмы* ★ Х/ф 'Переговорщик'.
AP@5	**0.20**	**0.30**	**0.28**	

Table 5b. Light viewer 6351697982861523430.

5. Discussion

5.1 Which recommendation strategy for this platform, and why?

Method	Slots already watched by user	Hits per user	Hits on watched	Precision, watched slots	Precision, new slots	Distinct programmes shown	Fallback slots
Popularity	67.2%	1.64	90.4%	44.1%	9.6%	5	0
MF	83.9%	1.90	95.2%	43.2%	11.3%	76	50
PPR	**92.8%**	**2.10**	**97.3%**	**44.1%**	**15.7%**	**199**	**50**
Own history	98.0%	2.06	99.1%	41.7%	19.0%	259	116

Table 4. List profile on the validation cohort (`results/analysis/list_profile.csv`). "Watched" = at least one full view by the same user in weeks 11–20; in the ground truth itself 56.7% of programmes are of this kind.

Familiar or new? Every list is mostly familiar, and the more familiar, the better the score: PPR fills 92.8% of its slots with programmes the user already watched, and 90–99% of every method's hits are such programmes. New slots rarely hit: 15.7% for PPR (only 7.2% of its slots) and 11.3% for MF. The two users show both sides. For the heavy viewer MF and PPR open with 'ТСН. Неделя' and 'Завтрак с 1+1' (popularity has them at ranks 1 and 5); PPR also keeps 'Говорит Украина' and 'Один за всех' from

his history, while MF spends slots on popular news he watched once or three times in training. No method finds ‘Касается каждого’, which he watched twice in training and five times later. For the light viewer four of five actual programmes are new to him (a crime documentary, two films, a news programme). Besides the familiar ‘Сегодня. Итоги’, all three methods catch only the popular ‘Факты. День’; none finds the films or the documentary, and whether he would have liked other new titles cannot be told from this log.

How the methods differ. *MF* learns shared co-viewing factors; with the strong regularisation selected (λ = 50) it pulls lists towards popular programmes (76 distinct programmes shown, 199 for PPR) and loses a user’s personal frequencies. *PPR’s diffusion* returns 30% of the mass to the user at every step (1 − α), so his heaviest edges dominate, and the walk lifts programmes that viewers sharing his edges watch: for the heavy user ‘Факты. День’ (3 training views) replaces ‘Великолепный век’ (13 views) at rank 4; for the light user ‘ТСН. Неделя’ and ‘Факты. День’ are new to him, and the latter is a hit (Fig. 3). *Popularity* shows the same five news programmes to everyone and already reaches MAP@5 0.2652: news is what most of these users watch. Of the secondary methods, association rules mainly encode weekly habit (A → A) for a few dozen frequent programmes, and content attributes are the only signal for the 3,787 test-window programmes nobody watched; as rankers both stayed behind PPR.



Fig. 3. PPR subgraphs of the two example users, 10 nodes each, mirrored (training weeks 11–20, α = 0.7, edge weight = full views): the seed, the 4 users with the highest PPR probability from it, and its PPR Top-5. For heavy viewer A four of the Top-5 are his own heaviest edges (58, 52, 17, 14 full views) and ‘Факты. День’ (3 views) is lifted by heavy co-viewers. For light viewer B two of the Top-5 have no edge from B and come only from the walk through other users; one of them, ‘Факты. День’, is a hit (`tools/ppr_two_user_graph.js`).

Recommended strategy. Select PPR with count-weighted edges (α = 0.7) as the main “your programmes this week” ranker, with the popularity list as the cold-start fallback; this is the designated final submission (a single method; no hybrid was submitted). It is the best method on validation (MAP@5 0.3595) and on Kaggle (public MAP@5 0.41930); its cost is modest at this scale — 55 s for 1,150 users with TF.js on 4 CPU cores, versus 133 s to train MF; and it always fills five personal slots for users with history, whereas the own-history list ran out for some users (116 fallback slots vs 50). The honest reading is that PPR is close to “repeat what the user watches”: for the user experience it is a reliable continue-watching shelf, not a discovery shelf. If the platform wants discovery, it should be a separate, labelled row judged by other evidence than MAP@5, because new-programme slots here hit at most 15.7% of the time.

Limitations supported by the results. Only 10.8% of schedule rows carry an id, and 47.6% of viewing time does not overlap a labelled airing, so no method can learn from it or recommend what was watched then. The log covers this platform only, so a programme absent from a user’s log may still have been seen elsewhere. Logged views drop sharply in weeks 21–27 (e.g. 105,223 in week 27 vs 278,122 in week 30), which thins the validation ground truth. Settings were selected on the same hold-out that reports them, so validation numbers are optimistic; Kaggle is the independent check. MAP@5 rewards repeats and says nothing about satisfaction; no online test was run and no business impact is claimed.

5.2 Failure case, root cause, fix

Failure. The first TF.js MF trained embeddings looked up with `tf.gather`. One gradient step did not finish in 100 s on the TF.js CPU backend, and the WASM backend stopped with `Kernel 'UnsortedSegmentSum' not registered`. **Root cause.** The gradient of `tf.gather` is a segment sum, which the WASM backend does not implement and the CPU backend computes very slowly. **Fix and verification.** The implicit-MF loss was rewritten over the full 4,737 × 2,500 user × programme matrix with `tf.matMul`, which has fast kernels on every backend; this is also the exact objective of [1]. Check: a self-check confirms that Adam lowers this loss, and the rewritten model reached validation MAP@5 0.3151 at k = 32, λ = 50. A second defect was found by a self-check: PPR leaked 7 tensors and MF 2 per run; wrapping intermediates in `tf.tidy` gave 0 leaks and the same MAP@5.

What worked. Edge weights for PPR (+0.09 MAP@5 over binary edges); the shared protocol, which made every difference attributable to the score function; next-week rules, which turn habit into rules. **What surprised me.** A list of the user’s own most-watched programmes (MAP@5 0.3552) is almost as good as PPR; content-based filtering is below popularity; Kaggle scores are higher than validation, but the method order is the same. **Next improvement.** A two-stage hybrid evaluated on the same hold-out: candidates from PPR, own history and the target-week schedule (how many times a programme airs), re-ranked with recency and airing-frequency features, as in [6].

6. AI Usage Disclosure

Tools. Claude (Anthropic) in the Claude desktop app, one agentic session linked to my computer (configured model identifier `claude-opus-5-5`); it edited files, ran Python/Node.js commands on my computer, ran headless-browser checks and searched the web. **How AI was used.** Code generation (all Python and JavaScript), debugging, data exploration, running the experiments, reference search, locating the lecture slide on edge weights, and writing the English text of this report from my decisions. English is not my first language, so I read every section and checked the numbers listed below instead of rewriting the text. I directed the work: I asked for the plan first, required TensorFlow.js instead of hand-written ML, each method in its own tab, the HW5 batch-size controls, the two-user graph view, user pickers, and a step-by-step submission builder for presenting; I asked why repeats are allowed and what the PPR parameters mean, and I pointed out that edge weights were mentioned in the lecture.

What I personally verified. (1) For user 15942687135322962396 I recomputed on a calculator the ground-truth tie-break and AP@5 = (1/1 + 2/2 + 3/3 + 4/5) / 5 = 0.76 from the CSV files (Section 4.4); it matches the value in `results/analysis/per_user_ap_valid.csv`. (2) I ran the application in my browser (WebGL), built and exported `submission_ppr.csv` and `submission_mf.csv` with it, and uploaded both to Kaggle myself; the returned scores are in Fig. 2.

What I trusted without verification. The mean over all 1,150 users (MAP@5 0.359528) and the other table values, computed by the code and not by me; the join code and the exploration numbers (checked only by the AI-written tests); the TF.js kernels; the implementation of each method beyond the automated checks; and the run times. **Session log.** The unmodified export of this session is submitted as `vladislav_kutranov_exam_session.json`; the session spans 22–25 October 2026 and was continued after automatic context summaries.

References

[1] Y. Hu, Y. Koren, and C. Volinsky, “Collaborative filtering for implicit feedback datasets,” in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, pp. 263–272, 2008. [Online]. Available: https://doi.org/10.1109/ICDM.2008.22. [Accessed: 2026-10-25].

[2] R. Agrawal and R. Srikant, “Fast algorithms for mining association rules,” in *Proc. 20th Int. Conf. Very Large Data Bases (VLDB)*, Santiago, Chile, pp. 487–499, 1994. [Online]. Available: https://www.vldb.org/conf/1994/P487.PDF. [Accessed: 2026-10-25].

[3] G. Jeh and J. Widom, “Scaling personalized web search,” in *Proc. 12th Int. World Wide Web Conf. (WWW)*, pp. 271–279, 2003. [Online]. Available: https://doi.org/10.1145/775152.775191. [Accessed: 2026-10-25].

[4] M. J. Pazzani and D. Billsus, “Content-based recommendation systems,” in *The Adaptive Web*, LNCS, vol. 4321, Springer, pp. 325–341, 2007. [Online]. Available: https://doi.org/10.1007/978-3-540-72079-9_10. [Accessed: 2026-10-25].

[5] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980. [Accessed: 2026-10-25].

[6] N. Melentev, “Two-layered recommender system methodology: a prize-winning solution,” *Towards Data Science*, Mar. 20, 2021. [Online]. Available: https://towardsdatascience.com/two-layered-recommender-system-methodology-a-prize-winning-solution-c4bdb815b156. [Accessed: 2026-10-25].

[7] TensorFlow Authors, “Release tfjs-v4.22.0,” GitHub, Oct. 2024. [Online]. Available: https://github.com/tensorflow/tfjs/releases/tag/tfjs-v4.22.0. [Accessed: 2026-10-25].

[8] ogurbych, “sweet.tv – TV program recommender,” Kaggle Community Prediction Competition, 2020. [Online]. Available: https://www.kaggle.com/competitions/sweettv-tv-program-recommender. [Accessed: 2026-10-25].

[9] S. Jin, “LLM4RecSys 06. Graph-based RecSys,” lecture slides (slides 3 and 5), LLM4Rec course, HSE University, 2026.